David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Subword and Character-Level Modeling

① Subword Motivation
② Writing Systems
③ Character Models
④ Pure Character CNNs
⑤ Subword Tokenization
⑥ Below-Word Modeling

Information from Subwords

- **Key Idea:**
  - Writing systems differ in how they segment and represent words.
  - Many NLP models benefit from analyzing **subword units** rather than whole words.
- **Why Subwords?**
  - Handles rare and out-of-vocabulary (OOV) words.
  - Improves generalization across morphologically rich languages.
  - Reduces the vocabulary size in neural models.
  - Helps in translation, speech recognition, and tokenization.

Information from Subwords

- **Examples of Subword Segmentation:**
  - **English (Morphologically Simple):** unhappiness → un + happiness
  - **German (Compound Words):** Donaudampfschifffahrtsgesellschaft → Donau + dampf + schiff + fahrt + gesellschaft (Danube Steamship Navigation Company)
  - **Japanese (Syllabic Writing):** 大学生 (university student) → 大 (dai, big) + 学 (gaku, study) + 生 (sei, student)
  - Arabic: ف+قال+نا+ها = فقلناها = so+said+we+it.

Writing System Variations & Subwords

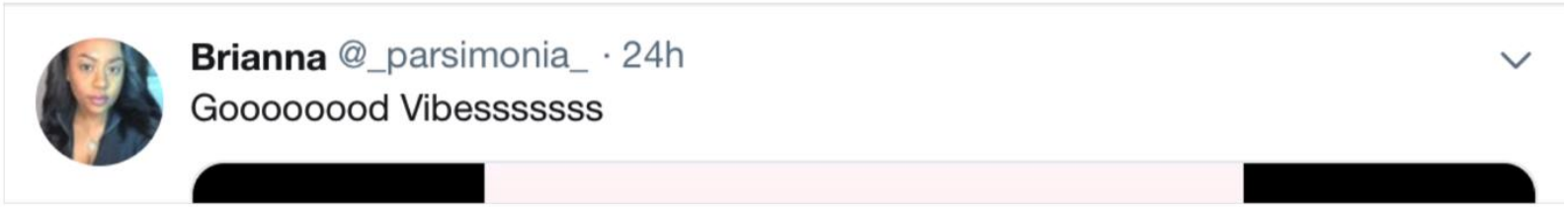
- **Writing Systems & Word Representation:**
  - **Alphabetic (English, Spanish):** Characters form words with clear spaces.
  - **Logographic (Chinese):** Characters often represent whole words or concepts.
  - **Abugida (Hindi, Tamil):** Symbols represent consonant-vowel pairs.
  - **Agglutinative (Turkish, Finnish):** Words are built by attaching multiple morphemes.
  - **Abjad (Arabic, Hebrew):** Letters only represent consonants, with vowels either not written

Writing System Variations & Subwords

- **Subword-Based NLP Methods:**
  - **Byte Pair Encoding (BPE):** Merges common character sequences (low + er → lower).
  - **WordPiece (Used in BERT):** Learns subwords based on frequency (play + ##ing → playing).
  - **Unigram Language Model:** Probabilistically selects optimal subword units.
  - **Character-Level Models:** Use individual characters to generate words dynamically, useful for handling OOV words.
- **Implications for NLP:**
  - Subword units balance word integrity and efficiency.
  - They enable multilingual models to share linguistic patterns.
  - They aid in better handling of spelling variations and new words.

Why Model Below the Word Level?

- Handling words as atomic units is problematic for languages with:
  - **Large, Open Vocabulary:** Too many unique words to store in a fixed vocabulary (e.g., names, rare words).
  - **Rich Morphology:** Many word variations due to prefixes, suffixes, infixes (e.g., Arabic, Finnish, Turkish).
  - **Transliteration:** Borrowed words with multiple spellings (e.g., "Mohammed" vs. "Muhammad").
  - **Informal Spelling:** Variations in online/social media text (e.g., "ur" for "your", Arabic هلا vs. هلاااا).



Character-Level Models in NLP

- **Word embeddings can be composed from character embeddings**, enabling models to understand words based on their internal structure.
  - **Generates embeddings for unknown words**, handling cases where a word is not present in the training vocabulary.
  - **Similar spellings share similar embeddings**, capturing relationships between morphologically related words (e.g., "run," "running," "runner").
  - **Solves the Out-of-Vocabulary (OOV) problem**, making NLP systems more robust to rare or unseen words.

- **Connected languages can be processed as characters**, particularly useful for scripts like Arabic or Chinese, where words are written without clear spaces.
  - **Both character-based and word-based methods have proven to work very successfully!**
  - **Somewhat surprisingly, traditional linguistics did not consider phonemes/letters as semantic units**—yet deep learning models can **compose meaningful representations** from character sequences.

Purely Character-Level Models

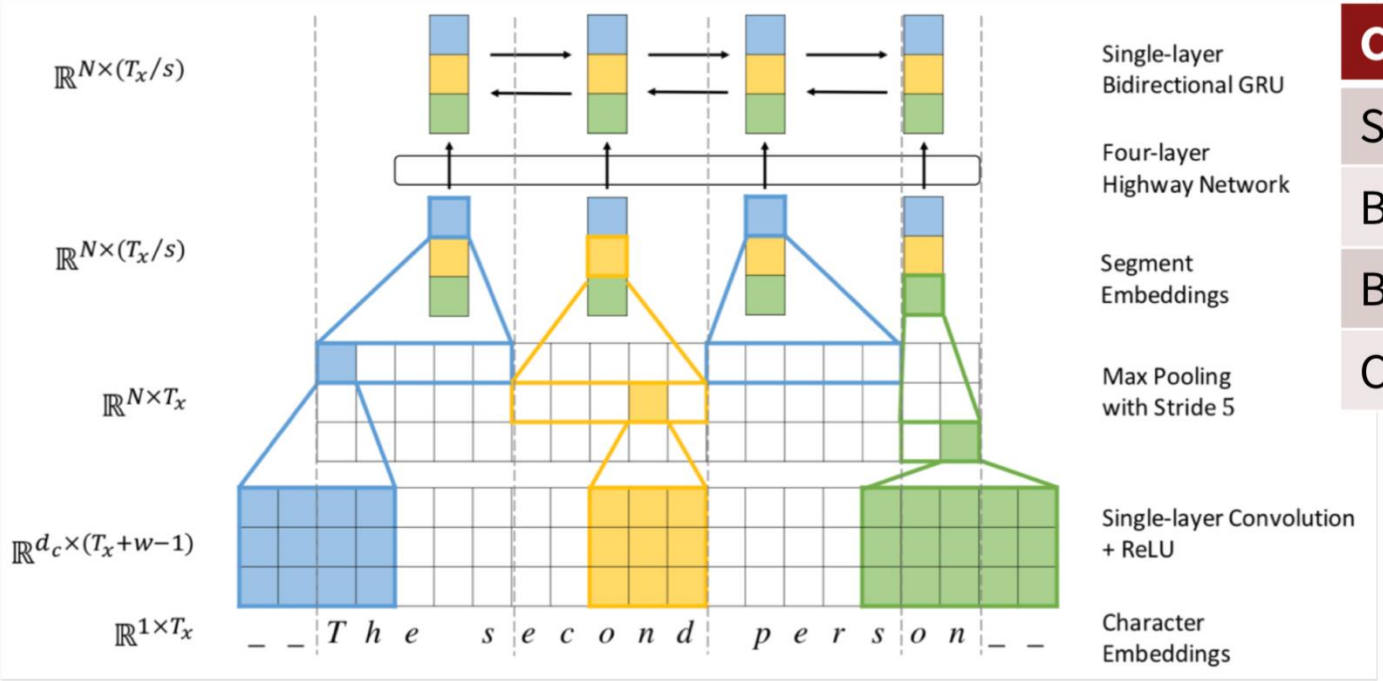
- **Character-Level Models in NLP**
  - Process text **directly at the character level**, without relying on predefined words or subwords.
  - Useful for **sentence classification, language modeling, and speech recognition**.
  - **Handles spelling variations, typos, and unseen words naturally.**

- **Example: Very Deep Convolutional Networks for Text Classification**
  - **Key Study: Very Deep Convolutional Networks for Text Classification**
  - Achieves **strong results** in text classification tasks **without word embeddings**.
  - Uses a **deep stack of convolutional layers**, similar to CNNs in vision tasks.
  - Learns hierarchical features **from characters to words to phrases to sentences**.

- **Why Purely Character-Based Models?**
  - Effective in **languages with complex morphology** (e.g., Finnish, Turkish, Arabic).
  - Works well for **low-resource languages** where word embeddings are unavailable.
  - More **robust to noise**, making them useful in user-generated content (social media, chats).

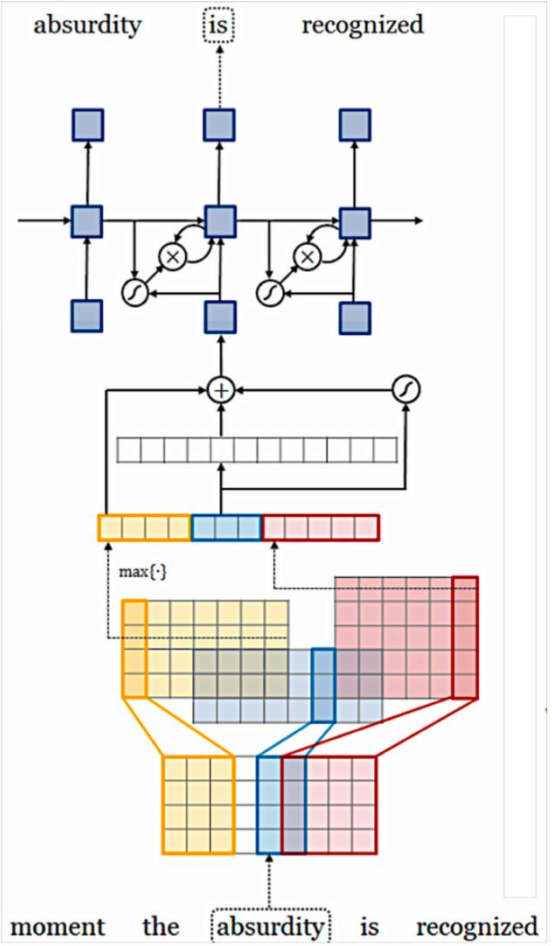
Fully Character-Level NMT

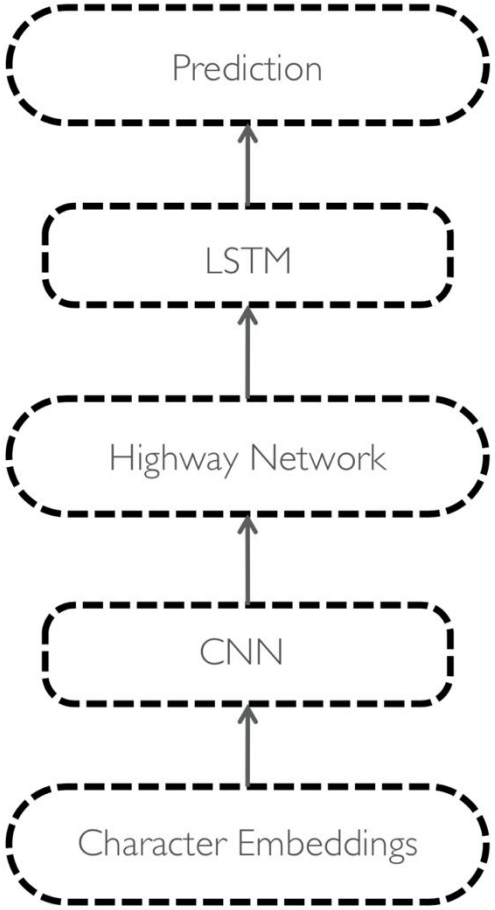
- Lee, Cho, and Hoffman (2017): Encoder (below) and decoder (character-level GRU) architecture.

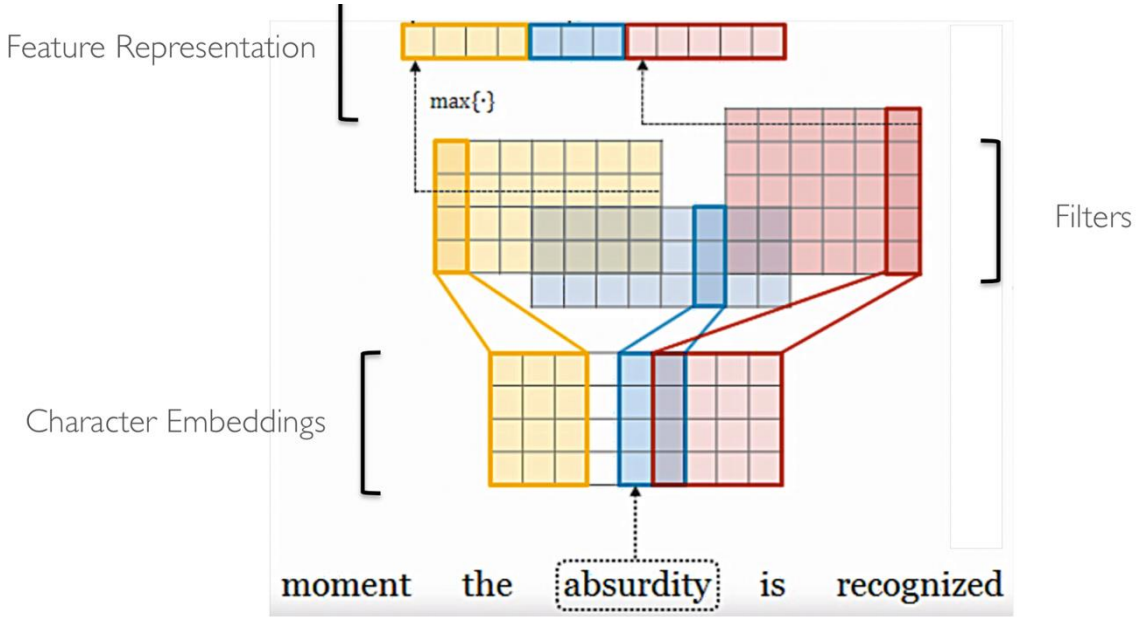


CS-En	WMT 15	Test
Source	Target	BLEU
Bpe	Bpe	20.3
Bpe	Char	22.4
Char	Char	22.5

Character-Level Neural LM







- Convolutions over character-level inputs.
- Max-over-time pooling (effectively n-gram selection).

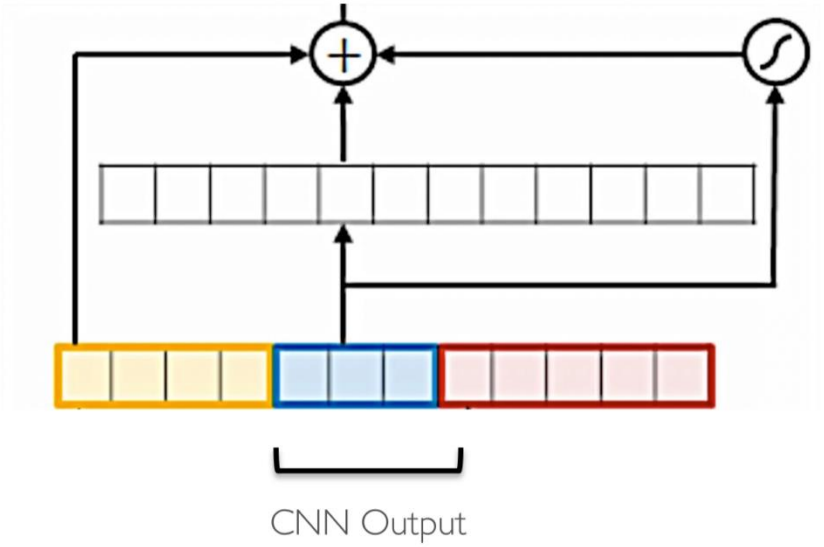
Character-Level Neural LM, Highway Layer

- Model *n*-gram interactions.
- Apply transformation while carrying over original information.
- Functions akin to an LSTM memory cell.

$$\mathbf{t} = \sigma(\mathbf{W}_T\mathbf{y} + \mathbf{b}_T)$$

$$\mathbf{z} = \mathbf{t} \odot g(\mathbf{W}_H\mathbf{y} + \mathbf{b}_H) + (\mathbf{1} - \mathbf{t}) \odot \mathbf{y}$$

Transform Gate Input Carry Gate



Byte-Pair Encoding (BPE)

- A **data compression-inspired** algorithm adapted for subword tokenization.
- Merges the **most frequent character sequences** iteratively to form subwords.
- Used in **NLP models like GPT and Transformer-based architectures** to handle unknown words and reduce vocabulary size.

Byte-Pair Encoding (BPE), Details

- **Initialize Vocabulary:** Start with individual characters as tokens.
  - Example: "low", "lowest" → {l, o, w, e, s, t}
- **Count Bigram Frequencies:** Find the most frequent adjacent character pairs.
  - Example: {(l, o): 5, (o, w): 4, (w, e): 3, (e, s): 2, (s, t): 2}
- **Merge the Most Frequent Pair:** Replace it with a new token.
  - Example: Merge (l, o) → "lo", updating the vocabulary.
- **Repeat Until Vocabulary Size is Reached:** Continue merging frequent pairs until a predefined vocabulary size is met.

Byte-Pair Encoding (BPE), Example

- **Example Evolution of BPE on "lowest":**
  - Step 1: {l, o, w, e, s, t}
  - Step 2: Merge **(l, o)** → **lo** → {lo, w, e, s, t}
  - Step 3: Merge **(lo, w)** → **low** → {low, e, s, t}
  - Step 4: Merge **(e, s)** → **es** → {low, es, t}
  - Step 5: Merge **(es, t)** → **est** → {low, est}

WordPiece

- **What is WordPiece Tokenization?**
  - A **subword tokenization method** used in models like **BERT**.
  - Designed to handle **rare words, OOV (out-of-vocabulary) words, and morphological variations**.
  - Uses a **data-driven approach** to break words into smaller, meaningful subword units based on frequency.

- **How WordPiece Works (Step-by-Step)**
  1. **Initialize Vocabulary:** Start with individual characters and a base vocabulary.
  2. **Compute Frequencies:** Find the most frequent subword pairs in a large corpus.
  3. **Merge the Most Frequent Pair:** Combine subwords iteratively to form larger units.
  4. **Continue Merging Until Vocabulary Size is Reached.**
  5. **Use '##' Prefix for Subword Units:** To indicate that a token is part of a previous word.

WordPiece, Example

- **WordPiece Tokenization of "unhappiness"**
  1. Start with individual characters: **u, n, h, a, p, p, i, n, e, s, s**
  2. Merge the most frequent pairs:
     1. "**un**" **+** "##**happiness**" → ["un", "##happiness"]
     2. "**hap**" **+** "##**piness**" → ["un", "##hap", "##piness"]
     3. "**happiness**" **splits further** → ["un", "##ha", "##pp", "##iness"]
  3. Final Tokenization:
     1. "**un**" "##**ha**" "##**pp**" "##**iness**"

Why Use WordPiece?

- Efficient vocabulary compression (common subwords are reused).
- Better handling of rare words and morphological variations.
- Improves multilingual models by sharing subwords across languages.
- Ensures minimal OOV issues in NLP applications.

Unigram LMs for Subword Tokenization

- **What is the Unigram Language Model?**
  - A probabilistic subword segmentation algorithm used in SentencePiece.
  - Unlike BPE and WordPiece, it does not rely on iterative merges.
  - Instead, it treats subwords as independent units and finds the most probable segmentation of a word.
- **How the Unigram Model Works (Step-by-Step)**
  1. Start with an initial vocabulary: Includes words and subwords.
  2. Assign probabilities to each subword: Based on frequency in a large corpus.
  3. Iterate by removing least probable subwords: Keep only the most informative ones.
  4. Segment words into the most likely subwords: Using a probabilistic model rather than deterministic merging.

Unigram LMs, Example and Benefits

- **Example: Tokenizing "unhappiness" with the Unigram Model**
  - Initial Vocabulary: {"un", "hap", "happiness", "##iness"}
  - Probabilities:
    - P("happiness") = 0.5, P("hap") = 0.3, P("##iness") = 0.2
  - Most Probable Segmentation:
    - **"un" + "hap" + "##iness"**

- **Why Use the Unigram Model?**
  - **More flexible than BPE/WordPiece:** Can represent multiple segmentations probabilistically.
  - **Better adaptation to different languages:** Works well with morphologically rich and script-heavy languages.
  - **Used in Google's SentencePiece tokenizer**, making it ideal for **unsupervised tokenization** in multilingual NLP models.